Material do Estudante — Desafio 5 — Isomorfismo

Estrutura de Dados II | Unidade III — Grafos | Profa. Kadidja Valéria

Nome: Arthur Santos Aguiar	**Turma:** N1	**Data:** 09/10/2026

Comparar os grafos **G**: V = {1, 2, 3, 4}; E = {(1,2), (2,3), (3,4), (4,1)} e **H**: V = {a, b, c, d}; E = {(a,c), (c,b), (b,d), (d,a)} e concluir se são isomorfos.

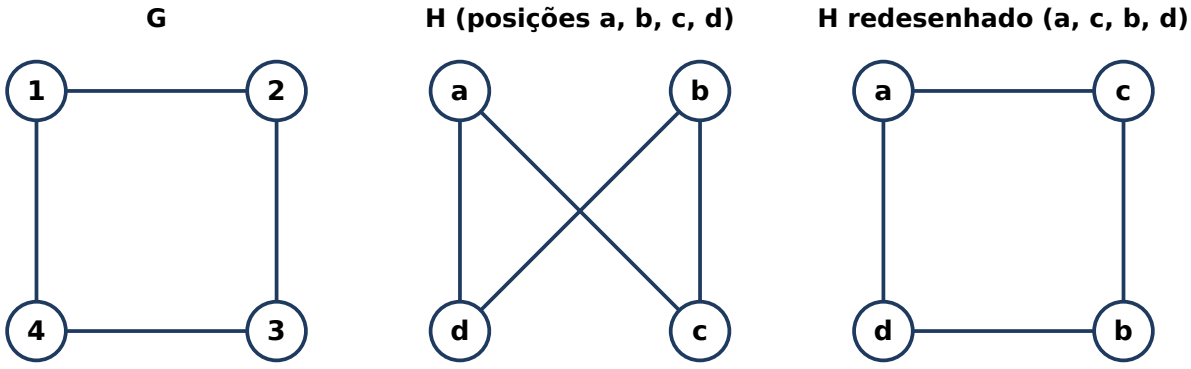


Figura 1 — Desenhado com os vértices na mesma posição, H parece uma “gravata-borboleta”; reposicionando c e b, ele assume exatamente a forma de G (um quadrado/ciclo).

1. Ordem e tamanho

Grafo	Ordem |V|	Tamanho |E|
G	4	4
H	4	4

Os dois grafos têm a mesma ordem e o mesmo tamanho — condição necessária para o isomorfismo.

2. Grau de cada vértice e sequência de graus

Vértice de G	Grau	Vértice de H	Grau
1 (vizinhos 2 e 4)	2	a (vizinhos c e d)	2
2 (vizinhos 1 e 3)	2	b (vizinhos c e d)	2
3 (vizinhos 2 e 4)	2	c (vizinhos a e b)	2
4 (vizinhos 3 e 1)	2	d (vizinhos b e a)	2

Sequência de graus de G: **(2, 2, 2, 2)** Sequência de graus de H: **(2, 2, 2, 2)**. Conferência: soma = 8 = 2 × 4 arestas nos dois grafos. Todo vértice tem grau 2, então ambos são 2-regulares.

3. Correspondência entre os vértices

Parti do vértice 1 → a. Como 1 é vizinho de 2, o correspondente de 2 precisa ser vizinho de a: c ou d. Escolhendo 2 → c, o vértice 3 (vizinho de 2) deve ir para o outro vizinho de c, que é b; sobra 4 → d. O programa em Java fez essa mesma busca por *backtracking* e encontrou **8 correspondências válidas** (as 8 simetrias do quadrado); a primeira delas é a da tabela.

Vértice em G	Correspondente em H
1	a
2	c
3	b
4	d

4. Verificação aresta por aresta

Aresta de G	Imagem f(u), f(v)	Pertence a E(H)?
(1,2)	(a,c)	Sim
(2,3)	(c,b)	Sim
(3,4)	(b,d)	Sim
(4,1)	(d,a)	Sim

As 4 arestas de G são levadas em 4 arestas **distintas** de H. Como |E(H)| = 4, todas as arestas de H são atingidas (o programa também confere no sentido inverso). Os pares não adjacentes também se preservam: em G, 1–3 e 2–4 não são arestas, e em H os pares correspondentes a–b e c–d também não são.

5. Conclusão e justificativa

G e H são isomorfos. A função f: V(G) → V(H) com f(1) = a, f(2) = c, f(3) = b, f(4) = d é uma bijeção (cada vértice de G corresponde a um único vértice de H e vice-versa) e satisfaz (u, v) ∈ E(G) ⇔ (f(u), f(v)) ∈ E(H). Os dois grafos são o ciclo C_4: a diferença está apenas nos rótulos e no desenho, não na estrutura.

Reflexão: ter a mesma sequência de graus é suficiente?

Não. Mesma ordem, mesmo tamanho e mesma sequência de graus são condições *necessárias*, mas não *suficientes*: servem para provar que dois grafos **não** são isomorfos (se diferirem), mas não provam que são. Contraexemplo: o ciclo C_6 e dois triângulos disjuntos têm 6 vértices, 6 arestas e sequência (2, 2, 2, 2, 2, 2), porém C_6 é conexo e o outro tem duas componentes (e possui triângulos, que C_6 não tem). O programa testou as 720 bijeções possíveis e não encontrou nenhuma que preserve as adjacências. Por isso, a confirmação do isomorfismo exige exibir a correspondência e verificar as arestas, como foi feito acima.

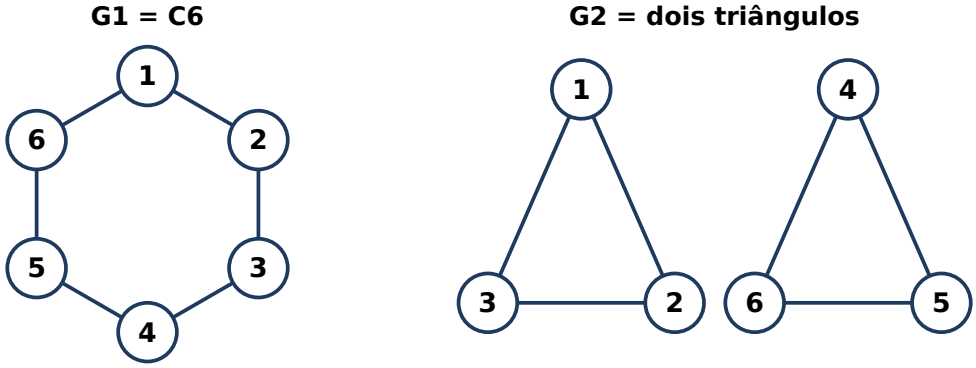


Figura 2 — Mesma sequência de graus (2, 2, 2, 2, 2, 2), mas estruturas diferentes.

Implementação em Java

Plataforma utilizada: **Java** (OpenJDK 21, compilado com `--release 8`, portanto roda em qualquer JVM a partir do Java 8). Arquivos entregues: `Desafio5Isomorfismo.java` (código-fonte) e `Desafio5Isomorfismo.class` (bytecode compilado).

Para executar: `java Desafio5Isomorfismo` na pasta do arquivo .class (ou recompilar com `javac Desafio5Isomorfismo.java`).

Saída do programa (execução real):

```
=== DESAFIO 5 - ISOMORFISMO ===
G: V = {1, 2, 3, 4}   E = {(1,2), (2,3), (3,4), (4,1)}
H: V = {a, b, c, d}   E = {(a,c), (c,b), (b,d), (d,a)}

=== 1. Ordem e tamanho ===
G: ordem |V| = 4   tamanho |E| = 4
H: ordem |V| = 4   tamanho |E| = 4
Mesma ordem? sim   Mesmo tamanho? sim

=== 2. Grau de cada vertice ===
G: d(1) = 2, d(2) = 2, d(3) = 2, d(4) = 2
H: d(a) = 2, d(b) = 2, d(c) = 2, d(d) = 2
Sequencia de graus de G: (2, 2, 2, 2)
Sequencia de graus de H: (2, 2, 2, 2)
Sequencias iguais? sim

=== 3. Busca de uma correspondencia (backtracking) ===
Correspondencias validas encontradas: 8
Correspondencia escolhida:
  Vertice em G | Correspondente em H
       1       |        a
       2       |        c
       3       |        b
       4       |        d

=== 4. Verificacao aresta por aresta ===
Ida (toda aresta de G vira aresta de H):
  (1,2) -> (a,c)  pertence a E(H)? sim
  (2,3) -> (c,b)  pertence a E(H)? sim
  (3,4) -> (b,d)  pertence a E(H)? sim
  (4,1) -> (d,a)  pertence a E(H)? sim
Volta (toda aresta de H vem de uma aresta de G):
  (a,c) <- (1,2)  pertence a E(G)? sim
  (c,b) <- (2,3)  pertence a E(G)? sim
  (b,d) <- (3,4)  pertence a E(G)? sim
  (d,a) <- (4,1)  pertence a E(G)? sim

=== 5. Conclusao ===
G e H SAO ISOMORFOS: a bijecao acima leva as 4 arestas de G
exatamente nas 4 arestas de H, preservando as adjacencias.
Os dois grafos sao o ciclo C4 com rotulos e desenhos diferentes.

=== Reflexao: mesma sequencia de graus basta? ===
G1 = C6 (ciclo de 6 vertices):  E = {(1,2), (2,3), (3,4), (4,5), (5,6), (6,1)}
G2 = dois triangulos disjuntos: E = {(1,2), (2,3), (3,1), (4,5), (5,6), (6,4)}
Sequencia de G1: (2, 2, 2, 2, 2, 2)
Sequencia de G2: (2, 2, 2, 2, 2, 2)
Correspondencias validas entre G1 e G2: 0 (de 720 bijecoes possiveis)
Logo, mesma sequencia de graus NAO garante isomorfismo:
G1 e conexo e G2 tem duas componentes; G2 tem triangulo e G1 nao.
```

Código-fonte: Desafio5Isomorfismo.java

```
/*
 * Estrutura de Dados II - Unidade III: Grafos
 * Profa. Kadidja Valeria
 *
 * Desafio 5 - Isomorfismo
 * Aluno: Arthur Santos Aguiar
 * Turma: N1
 * Data: 09/10/2026
 *
 * Ideia da solucao:
 *   - Representar G e H por matriz de adjacencia.
 *   - Comparar ordem |V|, tamanho |E| e sequencia de graus (invariantes).
 *   - Procurar, por backtracking, uma bijecao f: V(G) -> V(H) tal que
 *         (u,v) pertence a E(G)  <=>  (f(u), f(v)) pertence a E(H).
 *   - Conferir a correspondencia encontrada aresta por aresta.
 *   - Na reflexao, mostrar um contraexemplo: mesma sequencia de graus,
 *     mas grafos NAO isomorfos (C6 x dois triangulos).
 *
 * Compilar: javac Desafio5Isomorfismo.java
 * Executar: java Desafio5Isomorfismo
 */
import java.util.Arrays;

public class Desafio5Isomorfismo {

    // guarda a primeira correspondencia valida encontrada no backtracking
    static int[] primeiraCorrespondencia = null;

    public static void main(String[] args) {
        // ---------------- dados do enunciado ----------------
        String[] vG = {"1", "2", "3", "4"};
        String[][] eG = {{"1", "2"}, {"2", "3"}, {"3", "4"}, {"4", "1"}};

        String[] vH = {"a", "b", "c", "d"};
        String[][] eH = {{"a", "c"}, {"c", "b"}, {"b", "d"}, {"d", "a"}};

        int[][] g = montarMatriz(vG, eG);
        int[][] h = montarMatriz(vH, eH);

        titulo("DESAFIO 5 - ISOMORFISMO");
        System.out.println("G: V = " + conjunto(vG) + "   E = " + conjuntoArestas(eG));
        System.out.println("H: V = " + conjunto(vH) + "   E = " + conjuntoArestas(eH));

        // ---------------- 1. ordem e tamanho ----------------
        titulo("1. Ordem e tamanho");
        System.out.println("G: ordem |V| = " + vG.length + "   tamanho |E| = " + eG.length);
        System.out.println("H: ordem |V| = " + vH.length + "   tamanho |E| = " + eH.length);
        boolean mesmaOrdem = vG.length == vH.length;
        boolean mesmoTamanho = eG.length == eH.length;
        System.out.println("Mesma ordem? " + simNao(mesmaOrdem) + "   Mesmo tamanho? " + simNao(mesmoTamanho));

        // ---------------- 2. graus ----------------
        titulo("2. Grau de cada vertice");
        int[] grausG = calcularGraus(g);
        int[] grausH = calcularGraus(h);
        imprimirGraus("G", vG, grausG);
        imprimirGraus("H", vH, grausH);
        String seqG = sequencia(grausG);
        String seqH = sequencia(grausH);
        System.out.println("Sequencia de graus de G: " + seqG);
        System.out.println("Sequencia de graus de H: " + seqH);
        System.out.println("Sequencias iguais? " + simNao(seqG.equals(seqH)));

        if (!mesmaOrdem || !mesmoTamanho || !seqG.equals(seqH)) {
            System.out.println("Algum invariante e diferente: os grafos NAO podem ser isomorfos.");
            return;
        }

        // ---------------- 3. correspondencia ----------------
        titulo("3. Busca de uma correspondencia (backtracking)");
        int n = vG.length;
        int[] f = new int[n];
        Arrays.fill(f, -1);
        primeiraCorrespondencia = null;
        int total = contarIsomorfismos(g, h, 0, f, new boolean[n]);
        System.out.println("Correspondencias validas encontradas: " + total);

        if (primeiraCorrespondencia == null) {
```

```
            System.out.println("Nenhuma bijecao preserva as adjacencias: G e H NAO sao isomorfos.");
            return;
        }
        int[] mapa = primeiraCorrespondencia;
        System.out.println("Correspondencia escolhida:");
        System.out.println("  Vertice em G | Correspondente em H");
        for (int i = 0; i < n; i++) {
            System.out.println("       " + vG[i] + "       |        " + vH[mapa[i]]);
        }

        // ---------------- 4. verificacao aresta por aresta ----------------
        titulo("4. Verificacao aresta por aresta");
        boolean preserva = true;
        System.out.println("Ida (toda aresta de G vira aresta de H):");
        for (String[] a : eG) {
            int u = indice(vG, a[0]);
            int v = indice(vG, a[1]);
            String x = vH[mapa[u]];
            String y = vH[mapa[v]];
            boolean ok = h[mapa[u]][mapa[v]] == 1;
            preserva = preserva && ok;
            System.out.println("  (" + a[0] + "," + a[1] + ") -> (" + x + "," + y + ")  pertence a E(H)? " + simNao(ok));
        }
        // inversa da correspondencia, para conferir no sentido contrario
        int[] inversa = new int[n];
        for (int i = 0; i < n; i++) {
            inversa[mapa[i]] = i;
        }
        System.out.println("Volta (toda aresta de H vem de uma aresta de G):");
        for (String[] a : eH) {
            int x = indice(vH, a[0]);
            int y = indice(vH, a[1]);
            boolean ok = g[inversa[x]][inversa[y]] == 1;
            preserva = preserva && ok;
            System.out.println("  (" + a[0] + "," + a[1] + ") <- (" + vG[inversa[x]] + "," + vG[inversa[y]] + ")  pertence
        }

        // ---------------- 5. conclusao ----------------
        titulo("5. Conclusao");
        if (preserva) {
            System.out.println("G e H SAO ISOMORFOS: a bijecao acima leva as 4 arestas de G");
            System.out.println("exatamente nas 4 arestas de H, preservando as adjacencias.");
            System.out.println("Os dois grafos sao o ciclo C4 com rotulos e desenhos diferentes.");
        } else {
            System.out.println("A correspondencia nao preserva as adjacencias.");
        }

        // ---------------- reflexao ----------------
        titulo("Reflexao: mesma sequencia de graus basta?");
        String[] v6 = {"1", "2", "3", "4", "5", "6"};
        String[][] c6 = {{"1", "2"}, {"2", "3"}, {"3", "4"}, {"4", "5"}, {"5", "6"}, {"6", "1"}};
        String[][] doisTriangulos = {{"1", "2"}, {"2", "3"}, {"3", "1"}, {"4", "5"}, {"5", "6"}, {"6", "4"}};
        int[][] m1 = montarMatriz(v6, c6);
        int[][] m2 = montarMatriz(v6, doisTriangulos);
        System.out.println("G1 = C6 (ciclo de 6 vertices):  E = " + conjuntoArestas(c6));
        System.out.println("G2 = dois triangulos disjuntos: E = " + conjuntoArestas(doisTriangulos));
        System.out.println("Sequencia de G1: " + sequencia(calcularGraus(m1)));
        System.out.println("Sequencia de G2: " + sequencia(calcularGraus(m2)));
        int[] f6 = new int[6];
        Arrays.fill(f6, -1);
        primeiraCorrespondencia = null;
        int total6 = contarIsomorfismos(m1, m2, 0, f6, new boolean[6]);
        System.out.println("Correspondencias validas entre G1 e G2: " + total6 + " (de 720 bijecoes possiveis)");
        System.out.println("Logo, mesma sequencia de graus NAO garante isomorfismo:");
        System.out.println("G1 e conexo e G2 tem duas componentes; G2 tem triangulo e G1 nao.");
    }

    // monta a matriz de adjacencia de um grafo nao dirigido
    static int[][] montarMatriz(String[] vertices, String[][] arestas) {
        int n = vertices.length;
        int[][] m = new int[n][n];
        for (String[] a : arestas) {
            int u = indice(vertices, a[0]);
            int v = indice(vertices, a[1]);
            m[u][v] = 1;
            m[v][u] = 1;
        }
        return m;
    }
```

```
// grau = soma da linha; um laco (diagonal) conta duas vezes
static int[] calcularGraus(int[][] m) {
    int n = m.length;
    int[] graus = new int[n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graus[i] += m[i][j];
        }
        graus[i] += m[i][i];
    }
    return graus;
}

/*
 * Backtracking: tenta associar o vertice i de G a cada vertice j de H ainda
 * livre. So continua se a escolha for compativel com as anteriores.
 * Retorna quantas bijecoes completas preservam as adjacencias.
 */
static int contarIsomorfismos(int[][] g, int[][] h, int i, int[] f, boolean[] usado) {
    int n = g.length;
    if (i == n) {
        if (primeiraCorrespondencia == null) {
            primeiraCorrespondencia = f.clone();
        }
        return 1;
    }
    int total = 0;
    for (int j = 0; j < n; j++) {
        if (!usado[j] && compativel(g, h, i, j, f)) {
            f[i] = j;
            usado[j] = true;
            total += contarIsomorfismos(g, h, i + 1, f, usado);
            usado[j] = false;
            f[i] = -1;
        }
    }
    return total;
}

// i -> j so e possivel se, para todo k ja mapeado, (i,k) em G <=> (j,f(k)) em H
static boolean compativel(int[][] g, int[][] h, int i, int j, int[] f) {
    if (g[i][i] != h[j][j]) {
        return false;
    }
    for (int k = 0; k < i; k++) {
        if (g[i][k] != h[j][f[k]]) {
            return false;
        }
    }
    return true;
}

static int indice(String[] vertices, String rotulo) {
    for (int i = 0; i < vertices.length; i++) {
        if (vertices[i].equals(rotulo)) {
            return i;
        }
    }
    throw new IllegalArgumentException("Vertice inexistente: " + rotulo);
}

static void imprimirGraus(String nome, String[] v, int[] graus) {
    StringBuilder sb = new StringBuilder(nome + ": ");
    for (int i = 0; i < v.length; i++) {
        sb.append("d(").append(v[i]).append(") = ").append(graus[i]);
        if (i < v.length - 1) {
            sb.append(", ");
        }
    }
    System.out.println(sb);
}

// sequencia de graus em ordem nao decrescente
static String sequencia(int[] graus) {
    int[] copia = graus.clone();
    Arrays.sort(copia);
    StringBuilder sb = new StringBuilder("(");
    for (int i = 0; i < copia.length; i++) {
        sb.append(copia[i]);
        if (i < copia.length - 1) {
```

```
                sb.append(", ");
            }
        }
        return sb.append(")").toString();
    }

    static String conjunto(String[] v) {
        return "{" + String.join(", ", v) + "}";
    }

    static String conjuntoArestas(String[][] e) {
        StringBuilder sb = new StringBuilder("{");
        for (int i = 0; i < e.length; i++) {
            sb.append("(").append(e[i][0]).append(",").append(e[i][1]).append(")");
            if (i < e.length - 1) {
                sb.append(", ");
            }
        }
        return sb.append("}").toString();
    }

    static String simNao(boolean b) {
        return b ? "sim" : "nao";
    }

    static void titulo(String t) {
        System.out.println();
        System.out.println("=== " + t + " ===");
    }
}
```